



باسمه تعالی

دانشگاه صنعتی شریف

دانشکده مهندسی صنایع

تحقیق در عملیات ۲

فاز دوم پروژه درس

حل مسئله با کدنویسی پایتون و تحلیل حساسیت

اعضای گروه

آوا صدیقی ۴۰۱۱۰۱۵۹

پردیس رحیمی نصر ۴۰۱۱۷۰۴۰۱

شیده پرستاری فرکوش ۴۰۱۱۰۳۹۶۶

استاد

دکتر مریم رادمان

نیم سال دوم تحصیلی ۴۰۴-۱۴۰۳

فهرست

۳	مدل اصلاح شده
۳	پارامترها
۳	متغیرهای تصمیم‌گیری
۴	تابع هدف
۴	محدودیت‌ها
۶	حل مدل با pyomo
۶	تعریف مدل و مجموعه‌ها
۶	خواندن داده‌ها
۶	تعریف پارامترها
۷	متغیرهای تصمیم
۷	تابع هدف
۸	محدودیت‌ها
۱۰	حل مدل
۱۰	حل مدل با gurobipy
۱۲	دانلود دیتا فریم و نمایش بصری
۱۳	آزادسازی محدودیت اختصاص سه کشتی به اسکله سوم
۱۵	تحلیل حساسیت
۱۵	زمان بین تخلیه دو کشتی در اسکله شماره ۲
۱۶	زمان تخلیه کشتی شماره ۷ روی اسکله شماره ۳
۱۷	ظرفیت اسکله شماره ۲
۱۸	ظرفیت اسکله شماره ۳
۱۹	زمان رسیدن کشتی شماره ۶

در ابتدا اصلاحاتی به روی مدل ساخته شده در فاز ۱ اعمال می‌کنیم و سپس با مدل اصلاح شده در این فاز ادامه می‌دهیم. در این فایل متن مدل اصلاح شده، تفسیر کد حل آن با pyomo و gurobipy آورده شده است و پس از آن تحلیل حساسیت به روی مدل به تفصیل پیاده سازی شده است.

مدل اصلاح شده

پارامترها

C_i : مقدار بار کشتی i	C_j : ظرفیت بار اسکله j
l_i : طول کشتی i	L_j : طول اسکله j
t_{ij} : مدت زمانی که طول می‌کشد تا کشتی i در اسکله j تخلیه شود	p_i : اولویت کشتی i (بین ۰ و ۱)
C_{ECj} : هزینه استفاده از ظرفیت مازاد اسکله j	E_j : مقدار ظرفیت مازاد اسکله j
A_i : زمان ورود کشتی i تخلیه	ε_j : فاصله مورد نیاز اسکله j بین هر دو

متغیرهای تصمیم‌گیری

W_i : مدتی که کشتی i منتظر می‌ماند.

$$x_{ij} \begin{cases} 1 & \text{if ship } i \text{ is assigned to berth } j \\ 0 & \text{otherwise} \end{cases}$$

$$y_{ik} \begin{cases} 1 & \text{if } p_i > p_k \\ 0 & \text{if } p_k > p_i \end{cases} \quad i, k \in [1, 8], i < k$$

u_j : مقداری ظرفیت مازادی که در اسکله j مصرف می‌شود.

$$\delta_{1i} \begin{cases} 1 & \text{if } W_i \geq 10 \\ 0 & \text{otherwise} \end{cases}$$

$$\delta_{2i} \begin{cases} 1 & \text{if } W_i \geq 40 \\ 0 & \text{otherwise} \end{cases}$$

$$d_{1i} \begin{cases} W_i & \text{if } W_i \geq 10 \\ 0 & \text{otherwise} \end{cases}$$

$$d_{2i} \begin{cases} W_i & \text{if } W_i \geq 40 \\ 0 & \text{otherwise} \end{cases}$$

تابع هدف

$$\text{Min } Z = C_{\text{unload}} + C_W + C_{EC}$$

$$C_{\text{unload}} = \sum_j \sum_i 5 x_{ij} t_{ij}$$

$$C_{EC} = \sum_j u_j C_{EC_j}$$

$$\begin{aligned} C_W &= \sum_i C_{W_i} = \sum_i 25W_i + \delta_{1i}(-15W_i + 150) + \delta_{2i}(-5W_i + 200) \\ &= \sum_i 25W_i - 15 \delta_{1i}W_i + 150 \delta_{1i} - 5 \delta_{2i}W_i + 200 \delta_{2i} \\ &= \sum_i 25W_i - 15 d_{1i} + 150 \delta_{1i} - 5 d_{2i} + 200 \delta_{2i} \end{aligned}$$

محدودیت‌ها

- تخصیص یک کشتی به هر اسکله:

$$\sum_j x_{ij} = 1, \quad \forall i \in [1, 8]$$

- تعداد کشتی‌های تخصیص یافته به اسکله‌ها:

$$\sum_i x_{ij} \leq 2, \quad \forall j \in [1, 2, 4]$$

$$\sum_i x_{i3} = 3$$

- ظرفیت اسکله‌ها:

$$\sum_i x_{ij} c_i \leq C_j + u_j, \quad \forall j \in [1, 4]$$

- ظرفیت طول اسکله:

$$l_i x_{ij} \leq L_j, \quad \forall j \in [1, 4], \quad \forall i \in [1, 8]$$

- ترتیب و تخصیص کشتی‌ها:

$$p_i - p_k < y_{ik}, \quad i, k \in [1, 8], \quad i < k$$

$$p_k - p_i < 1 - y_{ik}, \quad i, k \in [1, 8], \quad i < k$$

- مدت انتظار کشتی‌ها:

$$W_k + A_k \geq W_i + A_i + t_{ij} + \varepsilon_j - M (1 - y_{ik} + 1 - x_{ij} + 1 - x_{kj})$$

$$j \in [1, 4], \quad i, k \in [1, 8], \quad i < k$$

$$W_i + A_i \geq W_k + A_k + t_{ij} + \varepsilon_j - M (y_{ik} + 1 - x_{ij} + 1 - x_{kj})$$

$$j \in [1, 4], \quad i, k \in [1, 8], \quad i < k$$

- حد بالای مدت انتظار کشتی‌ها:

$$W_i \leq 80 (1 - p_i), \quad \forall i \in [1, 8]$$

- δ_{2i} و δ_{1i} :

$$W_i - 10 \leq M\delta_{1i}, \quad i \in [1, 8]$$

$$W_i - 40 \leq M\delta_{2i}, \quad i \in [1, 8]$$

- خطی سازی تابع هدف:

$$d_{1i} \leq W_i, \quad i \in [1, 8]$$

$$d_{2i} \leq W_i, \quad i \in [1, 8]$$

$$d_{1i} \leq M\delta_{1i}, \quad i \in [1, 8]$$

$$d_{2i} \leq M\delta_{2i}, \quad i \in [1, 8]$$

حل مدل با pyomo

تعریف مدل و مجموعه‌ها

ابتدا یک مدل Pyomo از نوع ConcreteModel ایجاد می‌کنیم و دو مجموعه اصلی تعریف می‌کنیم.

- Ships: کشتی‌ها (۱ تا ۸)
- Berths: اسکله‌ها (۱ تا ۴)

خواندن داده‌ها

پارامترهای مدل از فایل Excel با نام Parameters.xlsx و از شیت‌های مختلف خوانده شده‌اند. سپس این داده‌ها به دیکشنری تبدیل شده‌اند تا به پارامترهای Pyomo تبدیل و استفاده شوند. برای t_{ij} می‌بایست توجه شود که هم مربوط به کشتی است و هم مربوط به اسکله و لازم است ابتدا به یک تاپل تبدیلش کنیم.

```
# loading excel sheets
c = pd.read_excel('Parameters.xlsx', sheet_name='Containers')
C = pd.read_excel('Parameters.xlsx', sheet_name='Capacity')
l = pd.read_excel('Parameters.xlsx', sheet_name='Length')
L = pd.read_excel('Parameters.xlsx', sheet_name='Max Length')
t = pd.read_excel('Parameters.xlsx', sheet_name='Unload Time')
p = pd.read_excel('Parameters.xlsx', sheet_name='Priority')
C_ec = pd.read_excel('Parameters.xlsx', sheet_name='Cost for Extra Capacity')
E = pd.read_excel('Parameters.xlsx', sheet_name='Max Extra Allowable Capacity')
A = pd.read_excel('Parameters.xlsx', sheet_name='Arrival Time')
epsilon = pd.read_excel('Parameters.xlsx', sheet_name='Lag')
anch = pd.read_excel('Parameters.xlsx', sheet_name='Anchorage')
```

تعریف پارامترها

تمام پارامترهای مسئله مانند ظرفیت، طول کشتی‌ها، زمان تخلیه، زمان ورود، اولویت، ظرفیت مازاد و هزینه‌ها با استفاده از Param تعریف می‌شوند. همانطور که پیش‌تر گفته شد برخی پارامترها مانند t (زمان تخلیه) به صورت دو بعدی (کشتی-اسکله) تعریف می‌شوند.

```
model_p.ShipBerths = Set(dimen=2, initialize=t_dict.keys())

# parameters
model_p.c = Param(model_p.Ships, initialize=c, mutable=True)
model_p.C = Param(model_p.Berths, initialize=C, mutable=True)
model_p.l = Param(model_p.Ships, initialize=l, mutable=True)
model_p.L = Param(model_p.Berths, initialize=L, mutable=True)
model_p.t = Param(model_p.ShipBerths, initialize=t_dict, mutable=True)
model_p.p = Param(model_p.Ships, initialize=p, mutable=True)
model_p.C_ec = Param(model_p.Berths, initialize=C_ec, mutable=True)
model_p.E = Param(model_p.Berths, initialize=E, mutable=True)
model_p.A = Param(model_p.Ships, initialize=A, mutable=True)
model_p.epsilon = Param(model_p.Berths, initialize=epsilon, mutable=True)
model_p.anch = Param(model_p.Berths, initialize=anch, mutable=True)
```

متغیرهای تصمیم

متغیرهای کلیدی مدل عبارتند از:

- $x[i, j]$: متغیر دودویی برای تخصیص کشتی i به اسکله j
- $y[i, k]$: متغیر ترتیب زمانی بین کشتی i و k
- $W[i]$: زمان انتظار کشتی i
- $u[j]$: ظرفیت مازاد اسکله j
- $d1[i], d2[i]$ و $delta1[i], delta2[i]$: متغیرهای کمکی برای مدل‌سازی هزینه‌های غیرخطی زمان انتظار

```
# if ship i assigned to berth j
model_p.x = Var(model_p.Ships, model_p.Berths, domain=Binary)

# if ship i is shipper than k
model_p.Y_index = Set(initialize=[(i, k) for i in model_p.Ships for k in model_p.Ships if i < k])
model_p.y = Var(model_p.Y_index, domain=Binary)

# waiting time of ship i
model_p.W = Var(model_p.Ships, domain=NonNegativeReals)

# used extra capacity
model_p.u = Var(model_p.Berths, domain=NonNegativeReals)

# if W[i] ≥ 10
model_p.delta1 = Var(model_p.Ships, domain=Binary)

# if W[i] ≥ 40
model_p.delta2 = Var(model_p.Ships, domain=Binary)

# if W[i] ≥ 10
model_p.d1 = Var(model_p.Ships, domain=NonNegativeReals)

# if W[i] ≥ 40
model_p.d2 = Var(model_p.Ships, domain=NonNegativeReals)
```

تابع هدف

تابع هدف مجموع سه بخش است:

- هزینه تخلیه (متناسب با زمان تخلیه)
- هزینه استفاده از ظرفیت مازاد
- هزینه زمان انتظار (تابع قطعه قطعه خطی)

```
model_p.objective = Objective(rule=objective_rule, sense=minimize)
```

محدودیت‌ها

- هر کشتی دقیقاً به یک اسکله تخصیص یابد.

```
# each ship is assigned to exactly one berth
def assign_one_berth(model_p, i):
    return sum(model_p.x[i, j] for j in model_p.Berths if (i, j) in model_p.ShipBerths) == 1
model_p.one_berth_per_ship = Constraint(model_p.Ships, rule=assign_one_berth)
```

- محدودیت تعداد کشتی در اسکله‌ها با توجه تعداد لنگرگاه‌ها

```
# number of ships assigned to berths
def max_two_ships_per_berth_1_and_2_and_4(model_p, j):
    if j in [1, 2, 4]:
        return sum(model_p.x[i, j] for i in model_p.Ships if (i, j) in model_p.ShipBerths) <= model_p.anch[j]
    else:
        return Constraint.Skip
model_p.max_ships_berth = Constraint(model_p.Berths, rule=max_two_ships_per_berth_1_and_2_and_4)

def exactly_three_ships_berth_3(model_p):
    return sum(model_p.x[i, 3] for i in model_p.Ships if (i, 3) in model_p.ShipBerths) == 3
model_p.exact_three_ships_berth3 = Constraint(rule=exactly_three_ships_berth_3)
```

- محدودیت طول کشتی نسبت به طول مجاز اسکله

```
# length constraint
def length_constraint(model_p, i, j):
    return model_p.l[i] * model_p.x[i, j] <= model_p.L[j]
model_p.length_constraint = Constraint(model_p.Ships, model_p.Berths, rule=length_constraint)
```

- محدودیت ظرفیت (با در نظر گرفتن ظرفیت مازاد)

```
# berths capacity
def berth_capacity_constraint(model_p, j):
    return sum(model_p.c[i] * model_p.x[i, j] for i in model_p.Ships if (i, j) in model_p.ShipBerths) <= model_p.c[j] + model_p.u[j]
model_p.capacity_constraint = Constraint(model_p.Berths, rule=berth_capacity_constraint)
```

- ترتیب تخلیه کشتی‌ها براساس اولویت

```
# Constraint: p[k] - p[i] <= y[i,k]
def priority_1(model_p, i, k):
    return model_p.p[i] - model_p.p[k] <= model_p.y[i, k]
model_p.priority_con_1 = Constraint(model_p.Y_index, rule=priority_1)

# Constraint: p[k] - p[i] <= 1 - y[i,k]
def priority_2(model_p, i, k):
    return model_p.p[k] - model_p.p[i] <= 1 - model_p.y[i, k]
model_p.priority_con_2 = Constraint(model_p.Y_index, rule=priority_2)
```

در این بخش بهتر است که محدودیت کوچکتر مساوی باشد اما از آنجایی که در pyomo نمی‌توان محدودیت اکید تعریف کرد و در داده‌های مسئله هیچ دو کشتی‌ای اولویت برابر ندارند تفاوتی ندارد و با کوچکتر مساوی این محدودیت را می‌نویسیم. از طرفی به همین دلیل در مدل سازی نیز برابر بودن اولویت دو کشتی را در نظر نگرفتیم.

- محدودیت‌های توالی ورود کشتی‌ها در اسکله مشترک

در این بخش دو محدودیت داریم که باهمدیگر کار می‌کنند. یا یکی از آن‌ها فعال است و یا هیچکدام. برای جلوگیری از ساخت محدودیت‌های اضافه و مازاد یک قید $k < i$ بر این سری محدودیت قرار می‌دهیم تا از بازنویسی محدودیت‌های تکراری خودداری کنیم.

```
# SEQUENCING 1
def sequencing_constraint_1(model_p, i, k, j):
    if i < k:
        return (
            model_p.W[k] + model_p.A[k] >=
            model_p.W[i] + model_p.A[i] + model_p.t[i, j] + model_p.epsilon[j]
            - M * (3 - model_p.y[i, k] - model_p.x[i, j] - model_p.x[k, j])
        )
    else:
        return Constraint.Skip
model_p.Sequencing1 = Constraint(model_p.Ships, model_p.Ships, model_p.Berths, rule=sequencing_constraint_1)

# SEQUENCING 2
def sequencing_constraint_2(model_p, i, k, j):
    if i < k:
        return (
            model_p.W[i] + model_p.A[i] >=
            model_p.W[k] + model_p.A[k] + model_p.t[i, j] + model_p.epsilon[j]
            - M * (model_p.y[i, k] + 2 - model_p.x[i, j] - model_p.x[k, j])
        )
    else:
        return Constraint.Skip
model_p.Sequencing2 = Constraint(model_p.Ships, model_p.Ships, model_p.Berths, rule=sequencing_constraint_2)
```

- محدودیت ماکسیمم W_i

```
# W[i] <= 80 * [1 - p[i]]
def waiting_upper(model_p, i):
    return model_p.W[i] <= 80 * (1 - model_p.p[i])
model_p.waiting_upper_con = Constraint(model_p.Ships, rule=waiting_upper)
```

کران بالای کلی برای زمان انتظار مازاد است به همین دلیل آن را تعریف نکردیم.

- خطی‌سازی تابع هدف

```
# Constraint: W[i] - 10 <= M * delta1[i]
def delta1_conditional_lower(model_p, i):
    return model_p.W[i] - 10 <= M * model_p.delta1[i]
model_p.delta1_lower_con = Constraint(model_p.Ships, rule=delta1_conditional_lower)

# Constraint: W[i] - 40 <= M * delta2[i]
def delta2_conditional_lower(model_p, i):
    return model_p.W[i] - 40 <= M * model_p.delta2[i]
model_p.delta2_lower_con = Constraint(model_p.Ships, rule=delta2_conditional_lower)
```

```
# Constraint: d1[i] <= W[i]
def d1_conditional_upper1(model_p, i):
    return model_p.d1[i] <= model_p.W[i]
model_p.d1_conditional_con1 = Constraint(model_p.Ships, rule=d1_conditional_upper1)

# Constraint: d2[i] <= W[i]
def d2_conditional_upper1(model_p, i):
    return model_p.d2[i] <= model_p.W[i]
model_p.d2_conditional_con1 = Constraint(model_p.Ships, rule=d2_conditional_upper1)
```

```
# Constraint: d1[i] <= M * delta1[i]
def d1_conditional_upper2(model_p, i):
    return model_p.d1[i] <= M * model_p.delta1[i]
model_p.d1_conditional_con2 = Constraint(model_p.Ships, rule=d1_conditional_upper2)

# Constraint: d2[i] <= M * delta2[i]
def d2_conditional_upper2(model_p, i):
    return model_p.d2[i] <= M * model_p.delta2[i]
model_p.d2_conditional_con2 = Constraint(model_p.Ships, rule=d2_conditional_upper2)
```

دقت داریم که از آنجایی که تابع هدف مینیم سازی است و ضریب پشت دو متغیر d_1 و d_2 منفی است، لازم نیست برای آن‌ها کران پایینی تعریف کنیم اگر کران پایین ۰ در تعریف آن‌ها وجود دارد. اما برای آنکه مسئله مقید شود لازم است کران بالا برای این دو متغیر تعریف شود.

حل مدل

در انتها مدل با استفاده از حل گر (solver) glpk حل می شود و خروجی ها چاپ می گردند.

```
from pyomo.opt import SolverStatus, TerminationCondition

solver = SolverFactory('glpk')
results = solver.solve(model_p, tee=True)
```

Optimal objective value: 1803.889016840444

```
Ship 1: Assigned to Berth 2
Ship 2: Assigned to Berth 4
Ship 3: Assigned to Berth 3
Ship 4: Assigned to Berth 1
Ship 5: Assigned to Berth 3
Ship 6: Assigned to Berth 2
Ship 7: Assigned to Berth 3
Ship 8: Assigned to Berth 4
```

```
Ship 1: 0.00 time units
Ship 2: 0.00 time units
Ship 3: 19.30 time units
Ship 4: 0.00 time units
Ship 5: 39.41 time units
Ship 6: 20.70 time units
Ship 7: 0.00 time units
Ship 8: 8.24 time units
```

مقدار بهینه تابع هدف را به همراه نحوه تخصیص و مقدار انتظار هر کشتی را پرینت می کنیم.
تفسیر دقیق تر معنای این جواب در بخش تحلیل حساسیت انجام خواهد شد اما پیش از آن به نحوه حل مدل با gurobipy می پردازیم.

حل مدل با gurobipy

در این بخش نیز مشابه پخش pyomo نیاز است پارامترها، متغیرها و محدودیت ها را به همراه تابع هدف تعریف کنیم.

پارامترها

```
c = pd.read_excel('Parameters.xlsx', sheet_name='Containers')
C = pd.read_excel('Parameters.xlsx', sheet_name='Capacity')
l = pd.read_excel('Parameters.xlsx', sheet_name='Length')
L = pd.read_excel('Parameters.xlsx', sheet_name='Max Length')
t = pd.read_excel('Parameters.xlsx', sheet_name='Unload Time')
epsilon = pd.read_excel('Parameters.xlsx', sheet_name='Lag')
p = pd.read_excel('Parameters.xlsx', sheet_name='Priority')
A = pd.read_excel('Parameters.xlsx', sheet_name='Arrival Time')
C_ec = pd.read_excel('Parameters.xlsx', sheet_name='Cost for Extra Capacity')
E = pd.read_excel('Parameters.xlsx', sheet_name='Max Extra Allowable Capacity')
anch = pd.read_excel('Parameters.xlsx', sheet_name='Anchorage')

t['Ship_Berth'] = t['Ship_Berth'].apply(eval)
t['Ship_Berth'] = t['Ship_Berth'].apply(lambda x: (int(x[0]), int(x[1])))

C_ec = dict(zip(C_ec['Berth'], C_ec['Cost for Extra Capacity']))
anch = dict(zip(anch['Berth'], anch['Anchorage']))
c = dict(zip(c['Ship'], c['Containers']))
C = dict(zip(C['Berth'], C['Capacity']))
E = dict(zip(E['Berth'], E['Max Extra Allowable Capacity']))
l = dict(zip(l['Ship'], l['Length']))
L = dict(zip(L['Berth'], L['Max Length']))
p = dict(zip(p['Ship'], p['Priority']))
A = dict(zip(A['Ship'], A['Arrival Time']))
epsilon = dict(zip(epsilon['Berth'], epsilon['Lag']))
t_dict = dict(zip(t['Ship_Berth'], t['Unload Time']))
```

متغیرها

```
# 1. x[i,j] - Ship i assigned to berth j
x = model.addVars(ships, berths, vtype=GRB.BINARY, name="x")

# 2. y[i,k] - Ship i scheduled before ship k (only if i < k)
Y_index = [(i, k) for i in ships for k in ships if i < k]
y = model.addVars(Y_index, vtype=GRB.BINARY, name="y")

# 3. u[j] - amount of extra capacity used by berth j
u = model.addVars(berths, vtype=GRB.CONTINUOUS, lb=0, name="u")
```

```
# Final Objective
model.setObjective(C_unload + C_W + C_EC, GRB.MINIMIZE)
```

محدودیت‌ها

دقیقا مشابه قبل محدودیت‌ها را تعریف می‌کنیم. برای نمونه چند محدودیتی که کمی متفاوت تعریف می‌شوند را به این بند اضافه می‌کنم.

محدودیت ترتیب کشتی‌ها

```
# Arrangement and allocation of ships
for i in ships:
    for k in ships:
        if i < k:
            model.addConstr(p[i] - p[k] <= y[i, k], name=f"order_1_{i}_{k}")
            model.addConstr(p[k] - p[i] <= 1 - y[i, k], name=f"order_2_{i}_{k}")
```

محدودیت زمان انتظار کشتی‌ها

```
# sequencing 1
for i in ships:
    for k in ships:
        if i < k:
            for j in berths:
                model.addConstr(
                    W[k] + A[k] >= W[i] + A[i] + t_dict[i, j] + epsilon[j] - M * (3 - y[i, k] - x[i, j] - x[k, j]),
                    name=f"order_1_{i}_{k}_{j}")

# sequencing 1
for i in ships:
    for k in ships:
        if i < k:
            for j in berths:
                model.addConstr(
                    W[i] + A[i] >= W[k] + A[k] + t_dict[i, j] + epsilon[j] - M * (2 + y[i, k] - x[i, j] - x[k, j]),
                    name=f"order_2_{i}_{k}_{j}")
```

همینطور، سایر محدودیت‌ها نیز به همین ترتیب نوشته می‌شوند و در نهایت با حل مدل به جوابی مشابه جواب بخش قبل می‌رسیم:

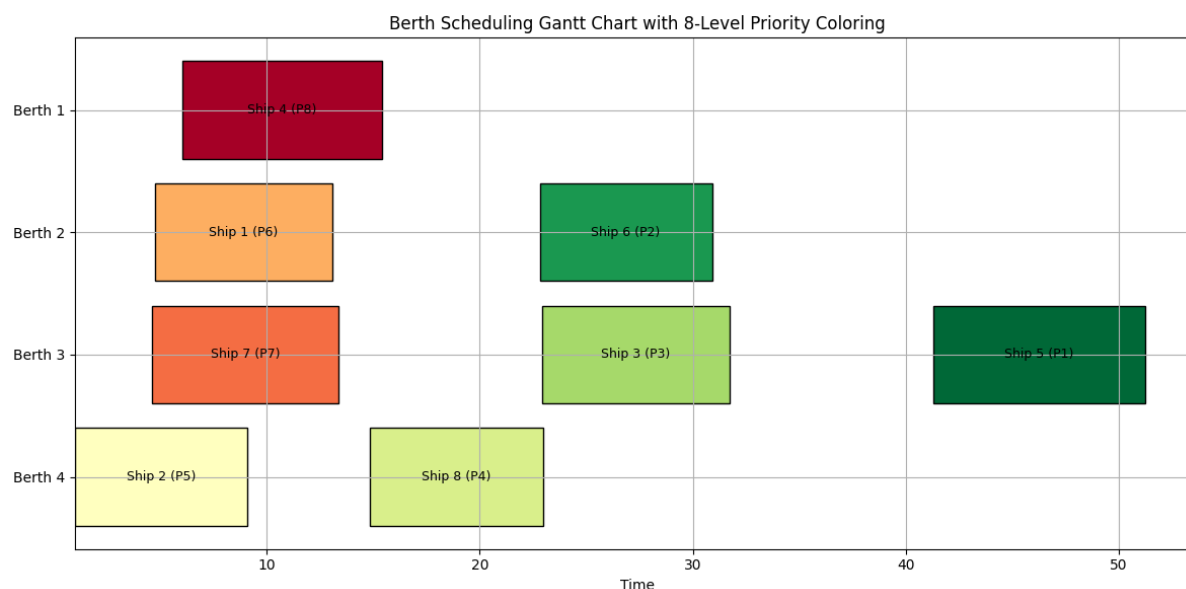
```
Optimal solution found (tolerance 1.00e-04)
Best objective 1.803889016840e+03, best bound 1.803889016840e+03, gap 0.0000%
Optimal solution found!

Ship assignments to berths:
Ship 1 assigned to Berth 2
Ship 2 assigned to Berth 4
Ship 3 assigned to Berth 3
Ship 4 assigned to Berth 1
Ship 5 assigned to Berth 3
Ship 6 assigned to Berth 2
Ship 7 assigned to Berth 3
Ship 8 assigned to Berth 4

Optimal objective function value: 1803.89
```

دانلود دیتا فریم و نمایش بصری

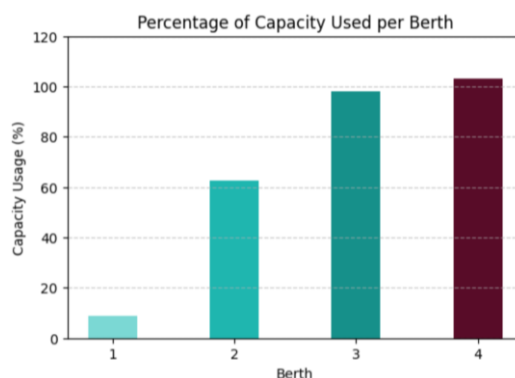
ابتدا با توجه به خواسته قسمت سوم، دیتافریم خواسته شده را می‌سازیم و دانلود می‌کنیم. فایل آن به فایل زیپ پیوست شده است. سپس مطابق آنچه خواسته شده است نمایشی بصری از آنچه در این بندرگاه اتفاق می‌افتد و تخصیص کشتی‌ها به اسکله‌ها را رسم می‌کنیم.



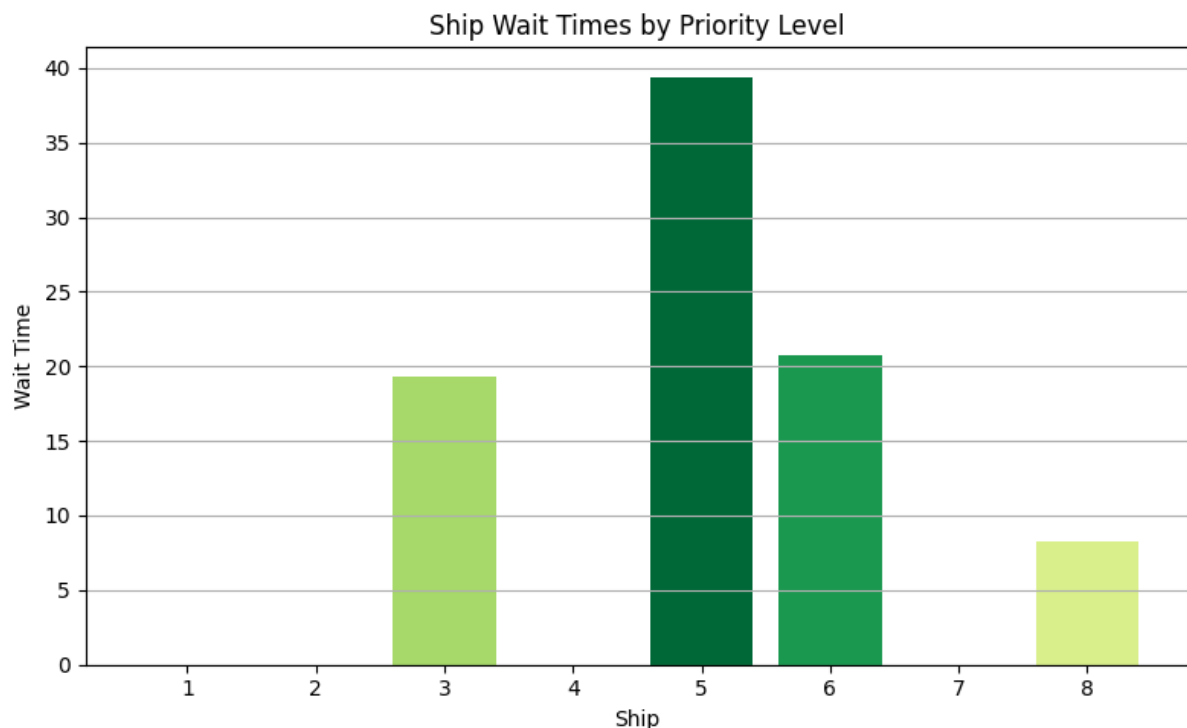
اولین نمودار مربوط به تخصیص هر کشتی به اسکله است و همینطور مدت زمان تخلیه بار آن کشتی. هر یک از مستطیل‌ها مدتی است که تخلیه بار به روی کشتی شماره i در حال انجام است. هرچقدر رنگ کشتی سبزتر است از اولویت کمتری برخوردار است و انتظار برای آن کشتی کم هزینه‌تر است. محور x نشان‌دهنده زمان است و محور y هر اسکله را نشان می‌دهد.

از نکاتی که در این نمودار به چشم می‌رسد می‌توان به آن اشاره کرد که همانطور که خیلی هم دور از انتظار نیست، مدل تصمیم می‌گیرد کشتی شماره ۴ که نسبتاً دیرتر می‌رسد و از همه کشتی‌ها از اولویت بیشتری برخوردار است را به اسکله‌ای جدا اختصاص دهد تا کمترین هزینه ممکن را ایجاد کند. از طرفی به اسکله ۳ که مدل ناچار است حتماً ۳ کشتی اختصاص دهد، دو تا از کم اولویت‌ترین کشتی‌ها را اختصاص می‌دهد.

دومین نمودار مربوط به ظرفیت مورد استفاده هر اسکله است. مشاهده می‌کنیم که اسکله ۳ مطابق خواسته مسئله تقریباً از تمام ظرفیت خود استفاده می‌کند. اسکله ۱ به دلیل اهمیت بسیار بالای کشتی ۴ که به آن اختصاص یافته است، از بخش کمی از ظرفیت خود استفاده می‌کند و اسکله ۴ ناچار است دست به استفاده از ظرفیت مازاد خود بزند. اسکله ۲ نیز چیزی در حدود ۶۰ درصد ظرفیت خود را استفاده می‌کند.



نمودار سوم نیز نشان‌دهنده آن است که کدام کشتی‌ها منتظر می‌مانند و چقدر منتظر می‌مانند. همانطور که انتظار داریم، بیشترین انتظار از آن کم اهمیت‌ترین کشتی خواهد بود و هرچه اولویت کشتی‌ها بالاتر می‌رود مقداری که منتظر می‌مانند نیز بیشتر می‌شود. کشتی‌هایی که در ۴ کشتی اولویت‌دار اول هستند نیز مطابق انتظارمان اصلاً منتظر نمی‌مانند.



آزادسازی محدودیت اختصاص سه کشتی به اسکله سوم

می‌دانیم که هرگاه مسئله‌ای را از بند محدودیتی آزاد می‌سازیم، تابع هدف مسئله بهتر می‌شود یا تغییری نمی‌کند. در این مورد تابع هدف مسئله به مقدار قابل ملاحظه‌ای بهتر می‌شود چراکه با تخصیص کشتی ۵ به اسکله ۱، زمان انتظار کشتی ۵ و به دنبال آن، هزینه‌های ناشی از آن به شکل چشمگیری کاهش می‌یابد.

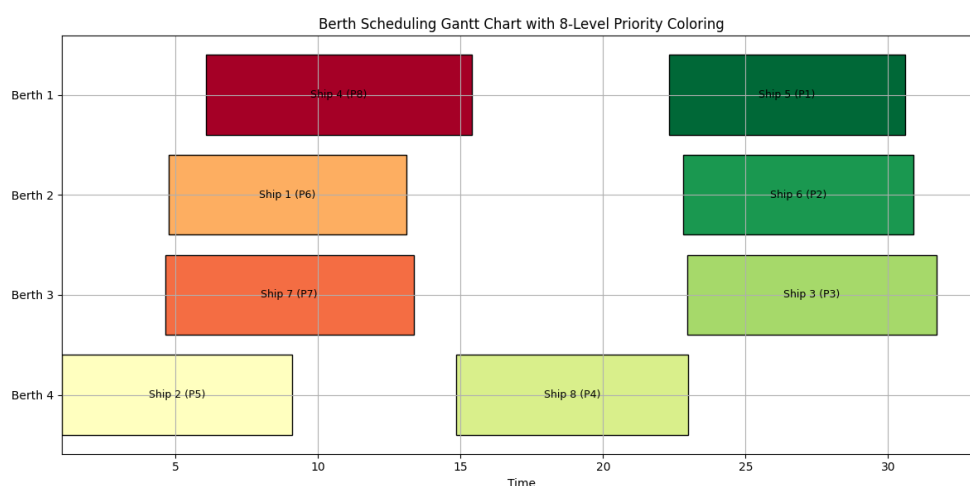
برای آزادسازی این محدودیت نیز باید فرم محدودیت قبلی به شکل زیر در آید:

```
def exactly_three_ships_berth_3(model_d):
    return sum(model_d.x[i, 3] for i in model_d.Ships if (i, 3) in model_d.ShipBerths) <= 3
model_d.exact_three_ships_berth3 = Constraint(rule=exactly_three_ships_berth_3)
```

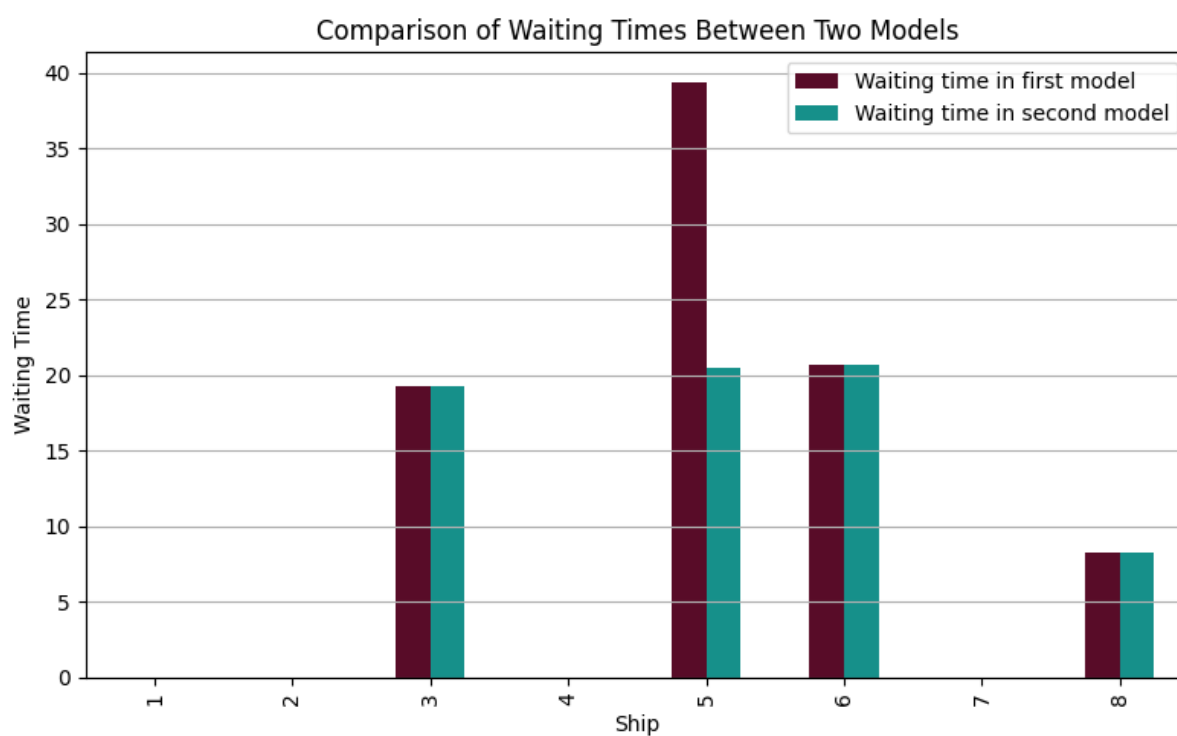
پس از اعمال این محدودیت در مسئله و حذف محدودیت قبلی، تابع هدف را خواهیم داشت که مقدار کمتری را نشان می‌دهد:

Original Objective: 1803.889016840444, New Objective: 1605.8666712510121

تخصیص جدید نیز به شکل زیر خواهد بود:



مشاهده می‌کنیم که تنها تخصیص کشتی ۵ تغییر کرده است اما همین تغییر باعث کاهش زمان انتظار برای این کشتی و کاهش تابع هدف خواهد شد.



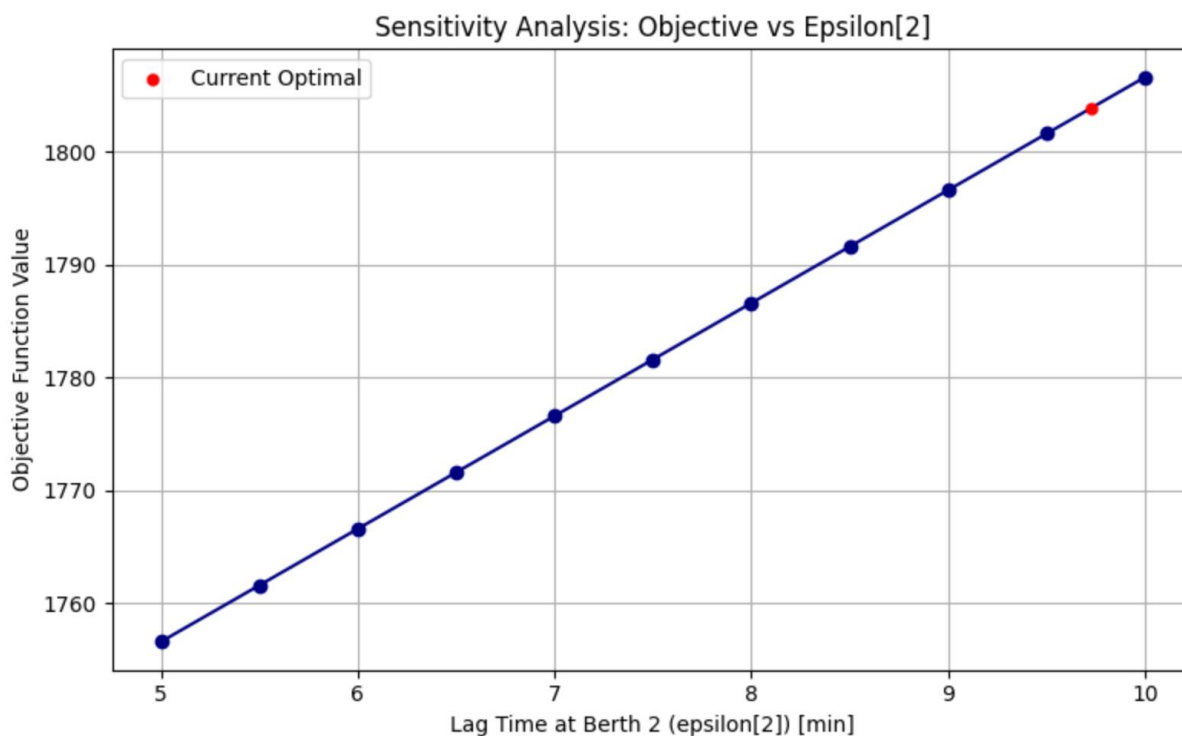
علاوه بر این، زمان تخلیه کشتی ۵ در اسکله ۱ نیز از زمان تخلیه این کشتی در اسکله ۳ کمتر است و این نکته نیز موجب کاهش دوباره تابع هدف در صورت عدم وجود محدودیت تخصیص ۳ کشتی به اسکله ۳ خواهد شد.

همچنین از آنجایی که تخصیص اسکله ۴ بدون تفاوت باقی می‌ماند اسکله ۴ همچنان نیاز به استفاده از ظرفیت مازاد خواهد داشت پس حذف این محدودیت کمکی به کاهش هزینه استفاده از ظرفیت مازاد نخواهد کرد.

در نهایت می‌توان گفت که حذف این محدودیت به کاهش ۲ بخش (هزینه زمان تخلیه و هزینه زمان انتظار) از ۳ بخش تابع هدف کمک خواهد کرد و در بخش هزینه ظرفیت مازاد تغییری ایجاد نخواهد کرد.

تحلیل حساسیت

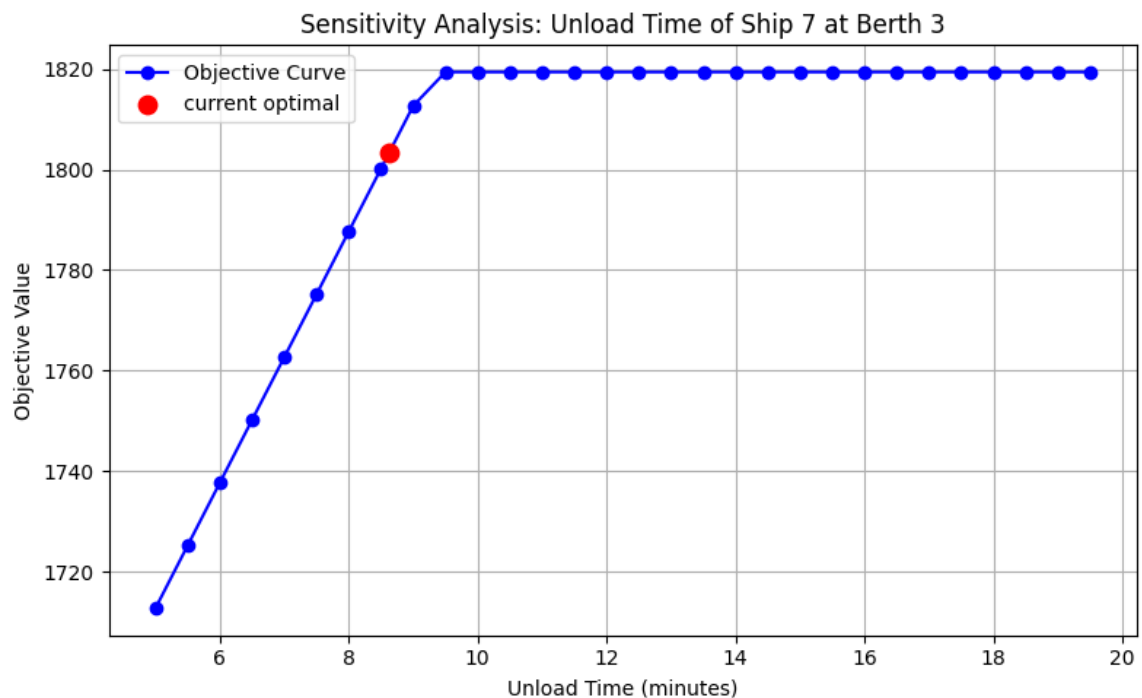
زمان بین تخلیه دو کشتی در اسکله شماره ۲



این نمودار تأثیر تغییرات پارامتر اپسیلون (زمان بین تخلیه دو کشتی) در اسکله ۲ بر روی تابع هدف نشان می‌دهد.

با افزایش ϵ از ۵ دقیقه تا ۱۰ دقیقه، تابع هدف به صورت خطی افزایش می‌یابد و به ازای هر ۰.۵ دقیقه افزایش، تابع هدف ۵ واحد افزایش می‌یابد. این نشان‌دهنده حساسیت بالا و مستقیم سیستم به زمان بین تخلیه کشتی‌ها است. واضح است که کاهش ϵ (زمان بین تخلیه کشتی‌ها) می‌تواند عملکرد سیستم را بهبود بخشد، اما ممکن است به منابع بیشتر نیاز داشته باشد.

زمان تخلیه کشتی شماره ۷ روی اسکله شماره ۳



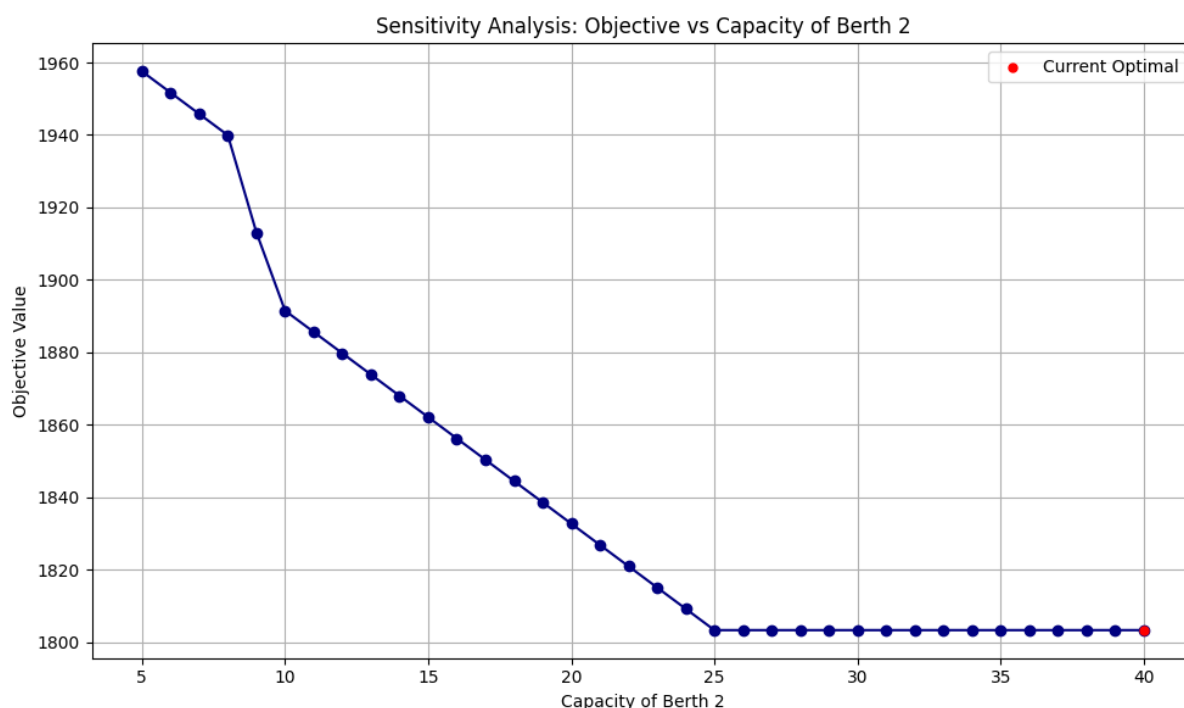
نمودار بالا تغییرات مقدار تابع هدف را بر اساس تغییر زمان تخلیه کشتی شماره ۷ در اسکله ۳ نمایش می‌دهد. زمان تخلیه در بازه‌ای بین ۵ دقیقه تا ۱۵ دقیقه با گام ۰.۵ دقیقه بررسی شده است.

از نمودار مشاهده می‌شود که با افزایش زمان تخلیه کشتی ۷، مقدار تابع هدف به صورت افزایشی رشد می‌کند؛ به این معنی که افزایش مدت زمان تخلیه باعث افزایش کلی هزینه‌ها یا زمان انتظار سیستم می‌شود. این روند افزایشی در ابتدا تقریباً خطی بوده و نشان‌دهنده حساسیت قابل توجه سیستم به این پارامتر مهم است.

با این حال، پس از رسیدن به یک مقدار خاص از زمان تخلیه (در حدود ۹.۵ دقیقه) نمودار به سمت یک حالت تقریباً ثابت و پایدار حرکت می‌کند. این ثبات نشان می‌دهد که پس از این مقدار، افزایش بیشتر زمان تخلیه تأثیر کمتری بر افزایش تابع هدف دارد و سیستم وارد یک مرحله از تحمل یا اشباع نسبت به این پارامتر می‌شود.

از این تحلیل می‌توان نتیجه گرفت که بهینه‌سازی زمان تخلیه کشتی‌ها، به ویژه کشتی شماره ۷، نقش بسیار مهمی در کاهش هزینه‌ها و بهبود کارایی سیستم دارد. کاهش زمان تخلیه تا مقدار بهینه می‌تواند عملکرد کلی سیستم را بهبود بخشد، ولی افزایش بیش از حد زمان تخلیه پس از نقطه ثبات، تأثیر زیادی بر سیستم نخواهد داشت.

ظرفیت اسکله شماره ۲

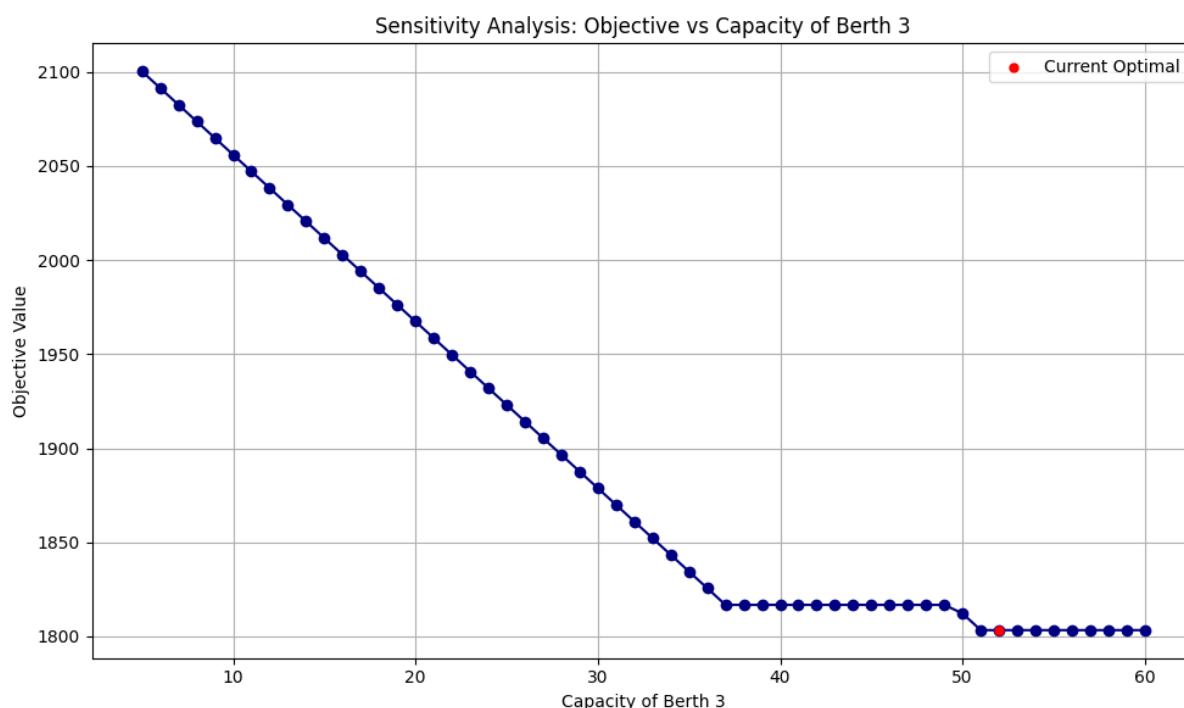


نمودار تحلیل حساسیت ظرفیت اسکله ۲ نشان‌دهنده تغییرات مقدار تابع هدف در پاسخ به افزایش ظرفیت اسکله از ۵ واحد تا ۴۰ واحد است.

در ابتدای نمودار، مشاهده می‌شود که با افزایش ظرفیت اسکله ۲، مقدار تابع هدف به شکل قابل توجهی کاهش می‌یابد. این روند نزولی بیانگر آن است که افزایش ظرفیت اسکله باعث کاهش هزینه‌ها یا زمان انتظار کشتی‌ها می‌شود و عملکرد کلی سیستم را بهبود می‌بخشد. علت این بهبود، افزایش توان عملیاتی اسکله و کاهش ترافیک و انتظار کشتی‌ها برای تخلیه است.

با این حال، پس از رسیدن ظرفیت اسکله به مقدار مشخصی (حدود ۲۵ واحد) نمودار تقریباً به حالت ثابت می‌رسد و افزایش بیشتر ظرفیت تأثیر چشمگیری بر بهبود تابع هدف ندارد. این نقطه نشان می‌دهد که پس از حد مشخصی، ظرفیت اسکله دیگر محدودیت اصلی سیستم نیست و سایر عوامل مانند زمان تخلیه یا ترتیب ورود کشتی‌ها نقش مهم‌تری در عملکرد کلی دارند.

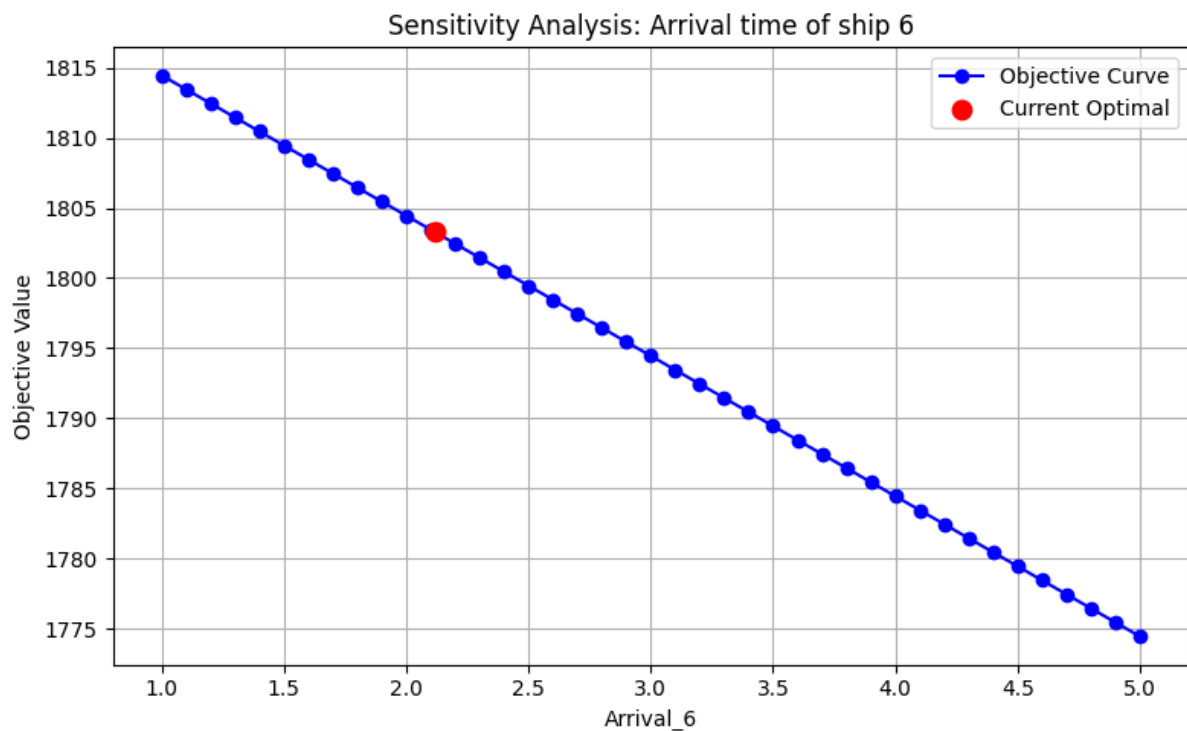
ظرفیت اسکله شماره ۳



نمودار تحلیل حساسیت ظرفیت اسکله ۳ تغییرات تابع هدف را در پاسخ به افزایش ظرفیت اسکله از ۵ تا ۶۰ واحد نمایش می‌دهد. البته طبق خواسته‌ی سوال تا ۴۰ واحد باید تغییر می‌کرد اما به علت اینکه نقطه بهینه فعلی در ۵۲ بود، بازه‌ی بزرگتری در نظر گرفته شد تا این مقدار مشخص شود.

در ابتدا با افزایش ظرفیت اسکله ۳، مقدار تابع هدف به شکل قابل توجهی کاهش می‌یابد. این کاهش بیانگر بهبود کارایی سیستم است زیرا ظرفیت بیشتر اسکله باعث کاهش زمان انتظار و بهبود تخصیص کشتی‌ها می‌شود. پس از این کاهش اولیه، نمودار به یک محدوده ثابت می‌رسد که در آن افزایش ظرفیت تأثیر چشمگیری بر مقدار تابع هدف ندارد. این نشان می‌دهد که در این بازه، ظرفیت اسکله محدودیت اصلی سیستم نیست و عوامل دیگر نقش پررنگ‌تری دارند. با افزایش بیشتر ظرفیت، مجدداً کاهش تابع هدف مشاهده می‌شود. این مرحله ممکن است نشان‌دهنده تغییرات در تخصیص کشتی‌ها یا بهبودهایی باشد که ظرفیت بالاتر اسکله فراهم می‌کند. در نهایت، نمودار دوباره به یک حالت ثابت می‌رسد که در آن ظرفیت‌های بسیار بالا تأثیر قابل توجهی بر بهبود تابع هدف ندارند.

زمان رسیدن کشتی شماره ۶



با افزایش زمان ورود از ۱ تا ۵ دقیقه، مقدار تابع هدف به صورت پیوسته و خطیکاهش می‌یابد. این کاهش با نرخ ثابت ۱ واحد در ازای هر ۰.۱ دقیقه افزایش زمان ورود اتفاق می‌افتد که نشان‌دهنده حساسیت و قابل پیش‌بینی بودن سیستم نسبت به این پارامتر است.

براساس نمودار می‌توان دریافت کرد که در این بازه کشتی ۶ هر چقدر دیرتر برسد، تابع هدف کمتر میشود و مطلوب‌تر خواهد بود.

نمودار حساسیت‌سنجی به وضوح نشان می‌دهد که با افزایش زمان ورود از ۱ به ۵ دقیقه، مقدار تابع هدف به طور کلی روند نزولی داشته است. این یافته حاکی از آن است که تأخیر کنترل‌شده در ورود کشتی ۶ می‌تواند به بهینه‌سازی عملکرد سیستم کمک کند.

با تشکر از توجه شما!